

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #7
CS@Cluj-Napoca

Agenda

- **Incomplete structures**
 - Review – meaning, usage, behavior
 - Incomplete Trees
- **Difference lists**

Incomplete structures - review

- Incomplete structures (lists, trees)
 - Replace the empty structure at the end **with a variable**
- Used as I/O structure (saves time and space)
- Change the behavior of regular predicates
- *Failure* conditions NEED to be:
 - ALL Explicit (no default failure in the absence of the clause)
 - Come FIRST (otherwise, is as if it does not exist)
- *Success* conditions should be:
 - All explicit (as before)
 - Start with the condition for the variable at the end
 - which comes right after the failure condition(s)

Search in BST

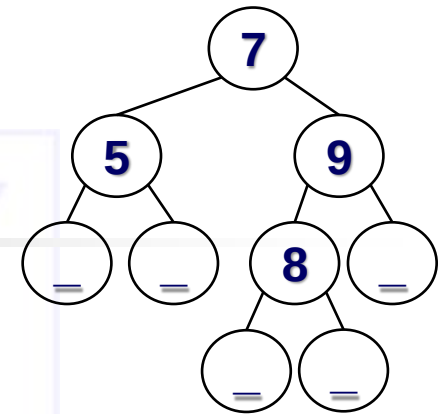
- This is an Incomplete (ending in variable) BST

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

```
search(K, t(K,_,_)):-!,
    write("found").
```

```
search(K, t(Key,Left,_)):-
    K<Key,!,
    search(K, Left).
```

```
search(K, t(_,_,Right)):-
    search(K, Right).
```



- The search predicate works well for a **regular** BST
- What behavior would it have in case of an incomplete tree?

Search in BST

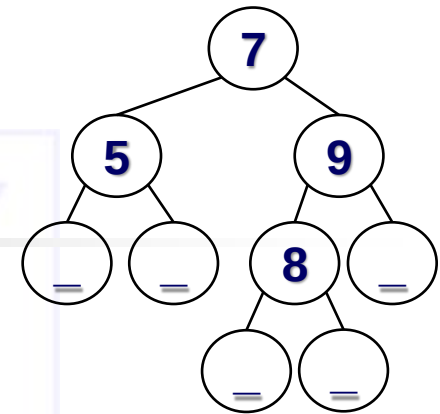
- It has **dual** significance in the search process:
 - if **K present**, reports so
 - else **inserts** it as leaf

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

```
search_ins(K,t(Key,_,_)):-!,
    write("found").
search_ins(K, t(Key,Left,_)):-
    K<Key,!,
    search_ins(K, Left).
search_ins(K, t(_,_,Right)):-
    search_ins(K, Right).
```

[q1] ?-tree(T),search_ins(9,T).
behaves as search (= **search_ins**)

[q2] ?-tree(T),search_ins(6,T).
behaves as insert (= **search_ins**)



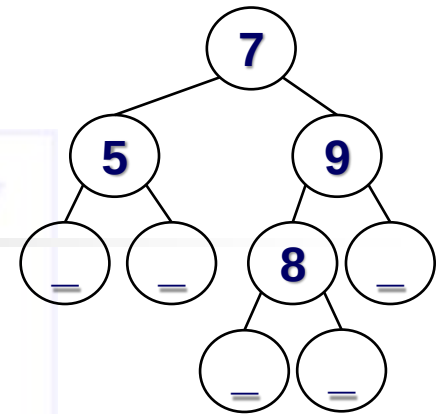
Search in BST

[q1] ?-tree(T), search_ins(9, T).
behaves as search (= **search_ins**)

```
search_ins(Key, t(Key, _, _)) :- !,
    write("found OR inserted").
```

The meaning of the clause is:

```
search_ins(Key, T) :-
    nonvar(T), T = t(Key1, _, _), Key1 = Key, !,
    write("found OR inserted").
```



Computer Science

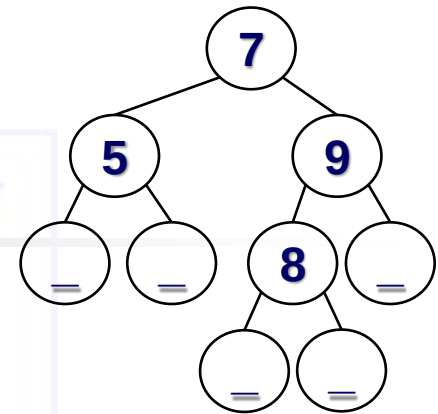
Search in BST

[q2] ?-tree(T), search_ins(6, T).
behaves as search (= search_ins)

```
search_ins(Key, t(Key, _, _)) :- !,
    write("found OR inserted").
```

The meaning of the clause is:

```
search_ins(Key, T) :-
    var(T), !, T=t(Key, _, _),
    write("found OR inserted").
```



PURE Search in BST (search by Key)

```
search_IT(K, T):-  
    var(T), !,  
    fail.  
search_IT(K, t(K,_,_)):-  
    !,  
    write("found").  
search_IT(K, t(Key,Left,_)):-  
    K<Key, !,  
    search_IT(K, Left).  
search_IT(Key, t(_,_,Right)):-  
    search_IT(K, Right).
```


Search in BST

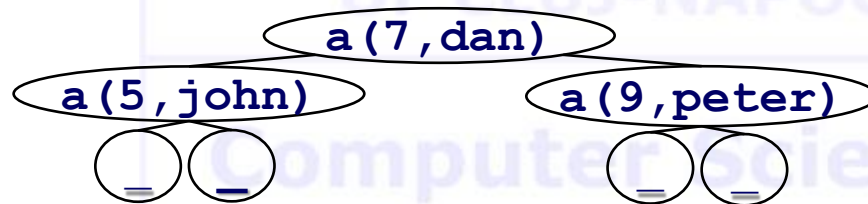
Assume the composite structure of type `a(key, value)`

The incomplete BST

Ex:

```
tree(
  n(
    n(
      n(
        n(
          a(5, john),
          a(7, dan),
          n(
            a(9, peter),
            _
          )
        )
      )
    )
  ).
```

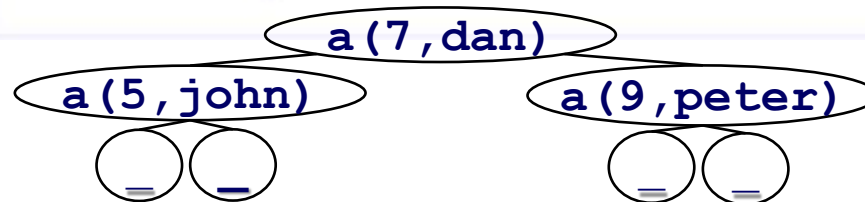
Represents the tree



Search in BST (search based on Key)

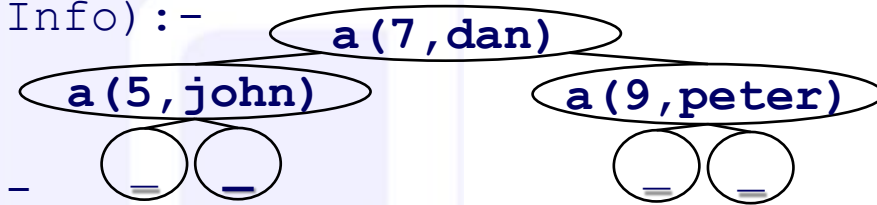
```
search_ins(K, n(_,a(K,Info),_), Info):-!.
search_ins(K, n(Left,a(Key,_),_), Info):-
    K<Key, !,
    search_ins(K, Left, Info).
search_ins(K, n(_,_,Right), Info):-
    search_ins(K, Right, Info).
```

[q1] ?- tree(T), **search_ins**(9,T,R).
Yes, R=peter

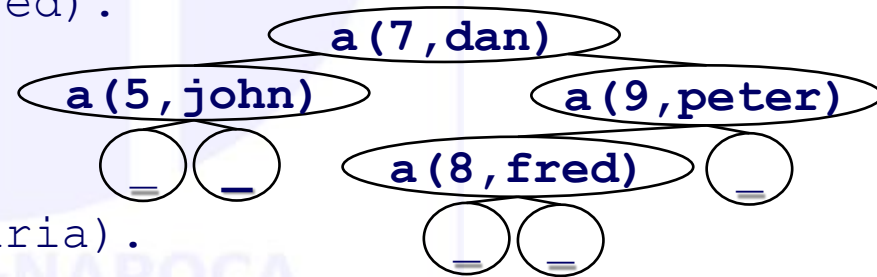


Search in BST (search based on Key)

```
search_ins(K, n(_, a(K, Info), _), Info) :- !.
search_ins(K, n(Left, a(Key, _), _), Info) :-
    K < Key, !,
    search_ins(K, Left, Info).
search_ins(K, n(_, _, Right), Info) :-
    search_ins(K, Right, Info).
```

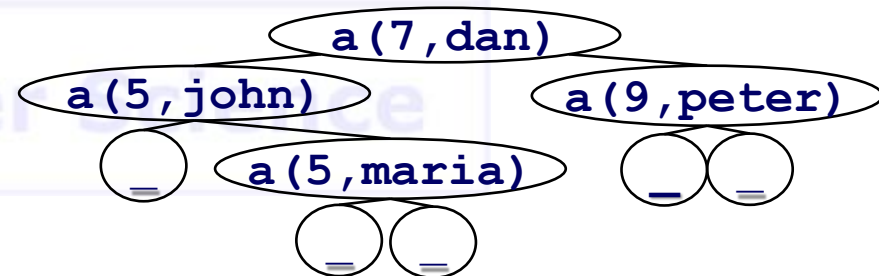


[q2] ?- tree(T), search_ins(8, T, fred).
Yes, T increases



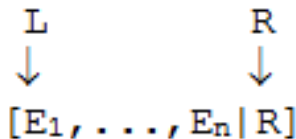
[q3] ?- tree(T), search_ins(5, T, maria).

- Does it fail?
 - If Yes, How?
 - If Not, Why? How is the tree affected?



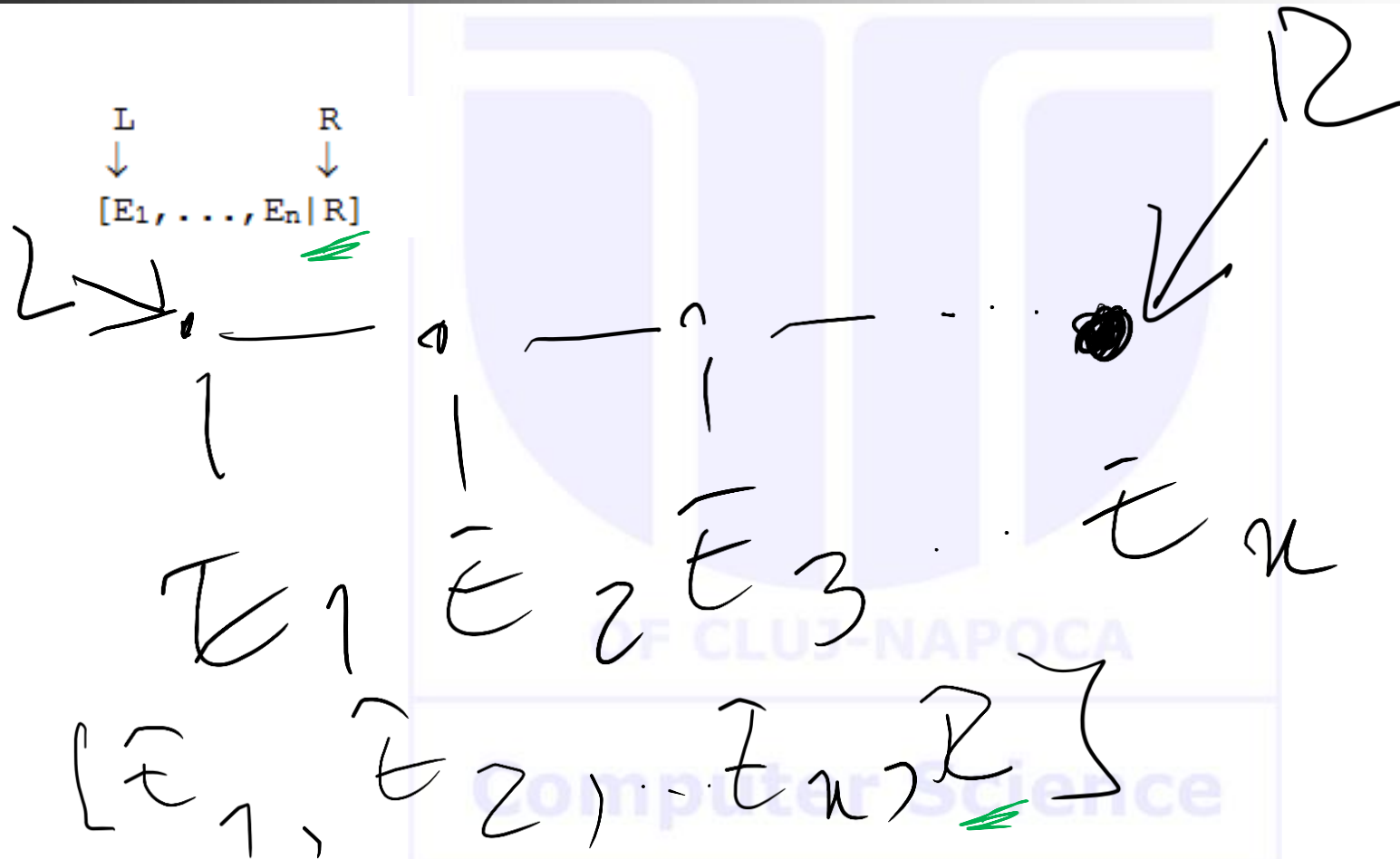
Difference Lists

- Another type of “irregular” structures
- Allow access to both the start and the END of lists 😊
- Named as lists with First and Last element
 - (that is, besides the variable =“pointer” at the beginning of the structure, we have a variable to “point” at the end of the list).
 - Similar to imperative languages where we have lists with pointers in the beginning and end of the list
- Allow for increased performance
- Difference list:



- D , denoted as $L-R$ (sometimes, in code, it is preferred to be represented as 2 arguments, L and R) is:
 - $D = L - R = [E_1, \dots, E_n]$
 - L “points” at the start, R at the “end”

Difference Lists



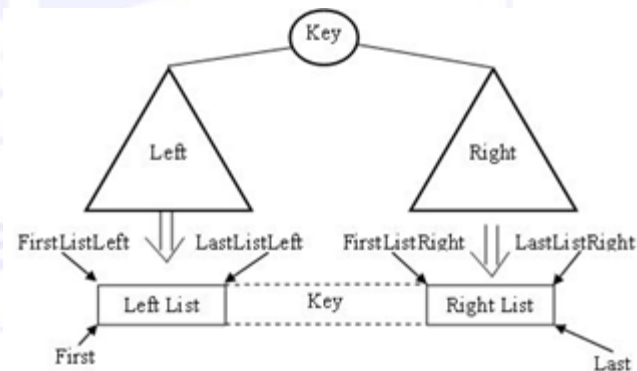
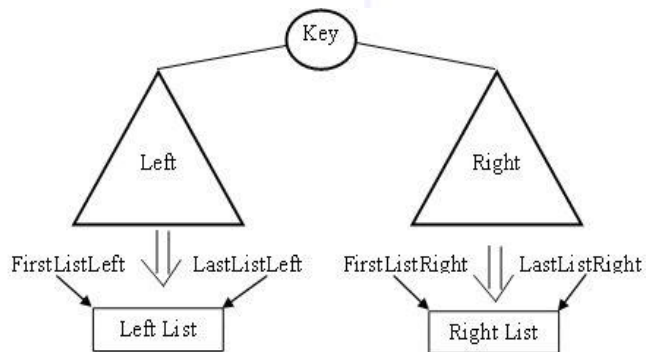
Tree to list (remember solutions in Lecture 5)

```
%generate_ordered_list/3(Tree, Start_diff_list, End_diff_list)
```

```
generate_preorder_list(nil, First, Last):-First=Last.
```

```
generate_preorder_list(t(Left,Key,Right),First,Last):-
```

```
    generate_preorder_list(Left, FirstListLeft, LastListLeft),
    generate_preorder_list(Right,FirstListRight,LastListRight),
    First= [Key|FirstListLeft],
    LastListLeft=FirstListRight,
    Last= LastListRight.
```



Tree 2 list forward – code (Lecture #5)

```
% sort_1/3(In_tree, Partial_result_list, Final_result_list)
```

```
sort_1(nil,L,L).
```

```
sort_1(n(Left,Key,Right), Lin, Lout):-  
    sort_1(Left, Lin, LLout),  
    append(LLout,[Key], LRin),  
    sort_1(Right, LRin, Lout).
```

```
sort_1(Tree,Result):-  
    sort_1(Tree,[],Result).
```

Computer Science

Tree to list

(same solution – check Lecture #5)

- Needs specialized call

```
generate_ordered_list(ITree, OList) :-  
    generate_ordered_list(ITree, OList, []).
```

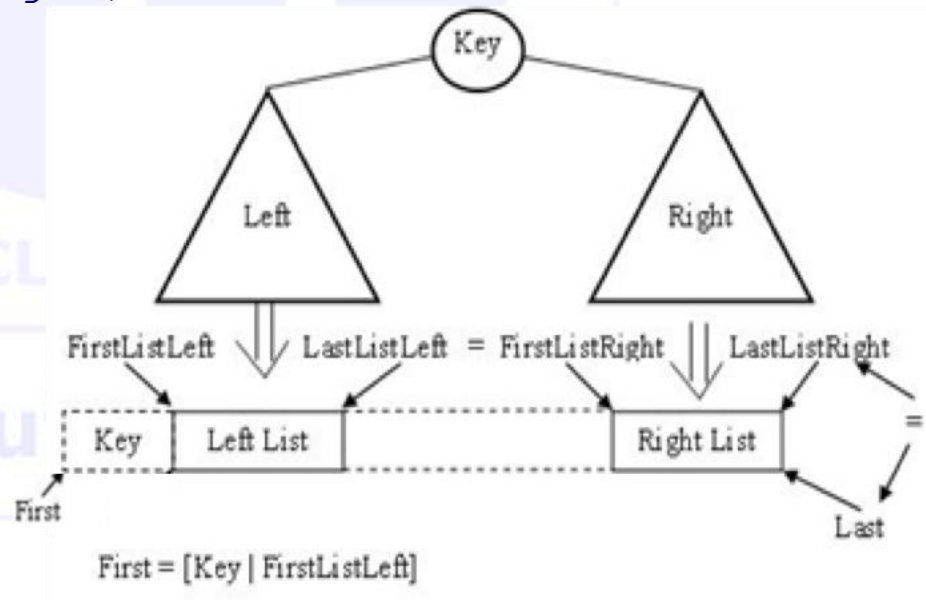
- Replace explicit unifications with implicit ones.

```
generate_ordered_list(nil, L, L).  
generate_ordered_list(t(Left, Key, Right), FirstListLeft, LastListRight) :-  
    generate_ordered_list(Left, FirstListLeft, [Key | Intermediate]),  
    generate_ordered_list(Right, Intermediate, LastListRight).
```

- Same as before, yet the utilization of the same name for variables (FirstListLeft) instead of making explicit unifications (First=FirstListLeft and the other 2).

Pre-order

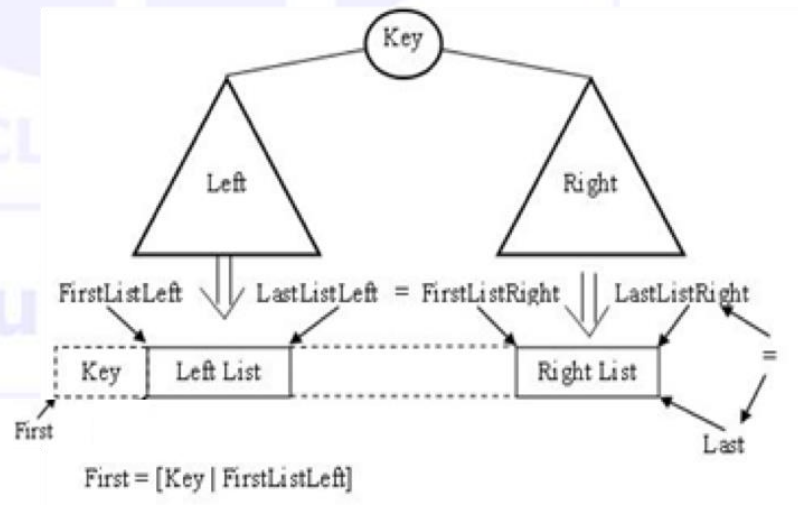
```
generate_preorder_list(nil, First, Last):-First=Last.
generate_preorder_list(t(Left,Key,Right),First,Last):-
    generate_preorder_list(Left,FirstListLeft,LastListLeft),
    generate_preorder_list(Right,FirstListRight,LastListRight),
    First= [Key|FirstListLeft],
    LastListLeft=FirstListRight,
    Last= LastListRight.
```



Pre-order

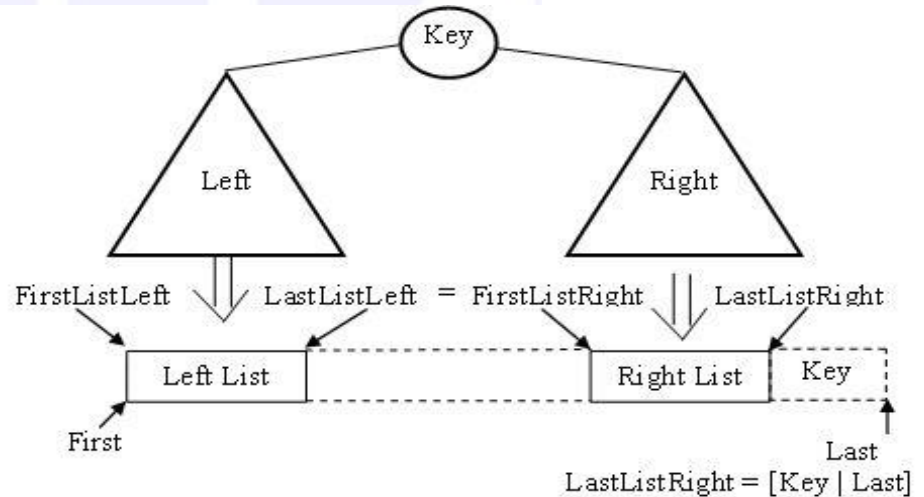
```
generate_preorder_list(nil,L,L).
generate_preorder_list(t(Left,Key,Right),[Key|FirstListLeft],LastListRight):-
    generate_preorder_list(Left,FirstListLeft,Interm),
    generate_preorder_list(Right,Interm,LastListRight).

generate_preorder_list(ITree,OList):-
    generate_preorder_list(ITree,OList,[]).
```



Post-order

```
generate_postorder_list(nil, First, Last):-First=Last.
generate_postorder_list(t(Left,Key,Right), First, Last):-
    generate_postorder_list(Left, FirstListLeft, LastListLeft),
    generate_postorder_list(Right,FirstListRight,LastListRight),
    First = FirstListLeft,
    LastListLeft = FirstListRight,
    LastListRight = [Key|Last].
```



Post-order

```
generate_postorder_list(nil,L,L).
generate_postorder_list(t(Left,Key,Right),FirstListLeft,LastListRight):-
    generate_postorder_list(Left,FirstListLeft,Intermediate),
    generate_postorder_list(Right,Intermediate,[Key|LastListRight]).

generate_postorder_list(ITree,OList):-
    generate_postorder_list(ITree,OList,[]).
```

